

Artificial Intelligence

Knowledge Representation, Inference and Reasoning

Chapter-4

Knowledge Representation

Humans are best at understanding, reasoning, and interpreting knowledge. Human knows things, which is knowledge and as per their knowledge they perform various actions in the real world. But how machines do all these things comes under knowledge representation and reasoning. Hence we can describe Knowledge representation as following:

- Knowledge representation and reasoning (KR, KRR) is the part of Artificial intelligence which concerned with AI agents thinking and how thinking contributes to intelligent behavior of agents.
- It is responsible for representing information about the real world so that a computer can understand and can utilize this knowledge to solve the complex real world problems such as diagnosis a medical condition or communicating with humans in natural language.
- It is also a way which describes how we can represent knowledge in artificial intelligence. Knowledge representation is not just storing data into some database, but it also enables an intelligent machine to learn from that knowledge and experiences so that it can behave intelligently like a human.

What to Represent:

Following are the kind of knowledge which needs to be represented in AI systems:

Object: All the facts about objects in our world domain.

E.g., Guitars contains strings, trumpets are brass instruments.

Events: Events are the actions which occur in our world.

Performance: It describe behavior which involves knowledge about how to do things.

Meta-knowledge: It is knowledge about what we know.

Facts: Facts are the truths about the real world and what we represent.

Knowledge-Base: The central component of the knowledge-based agents is the knowledge base. It is represented as KB. The Knowledgebase is a group of the Sentences (Here, sentences are used as a technical term and not identical with the English language).

Knowledge: Knowledge is awareness or familiarity gained by experiences of facts, data, and situations.

Following are the types of knowledge in artificial intelligence:



1. Declarative Knowledge:

- Declarative knowledge is to know about something.
- It includes concepts, facts, and objects.
- It is also called descriptive knowledge and expressed in declarative sentences.
- It is simpler than procedural language.

2. Procedural Knowledge

- It is also known as imperative knowledge.
- Procedural knowledge is a type of knowledge which is responsible for knowing how to do something.
- It can be directly applied to any task.
- It includes rules, strategies, procedures, agendas, etc.
- Procedural knowledge depends on the task on which it can be applied.

3. Meta-knowledge:

- Knowledge about the other types of knowledge is called Meta-knowledge.

4. Heuristic knowledge:

- Heuristic knowledge is representing knowledge of some experts in a field or subject.
- Heuristic knowledge is rules of thumb based on previous experiences, awareness of approaches, and which are good to work but not guaranteed.

5. Structural knowledge:

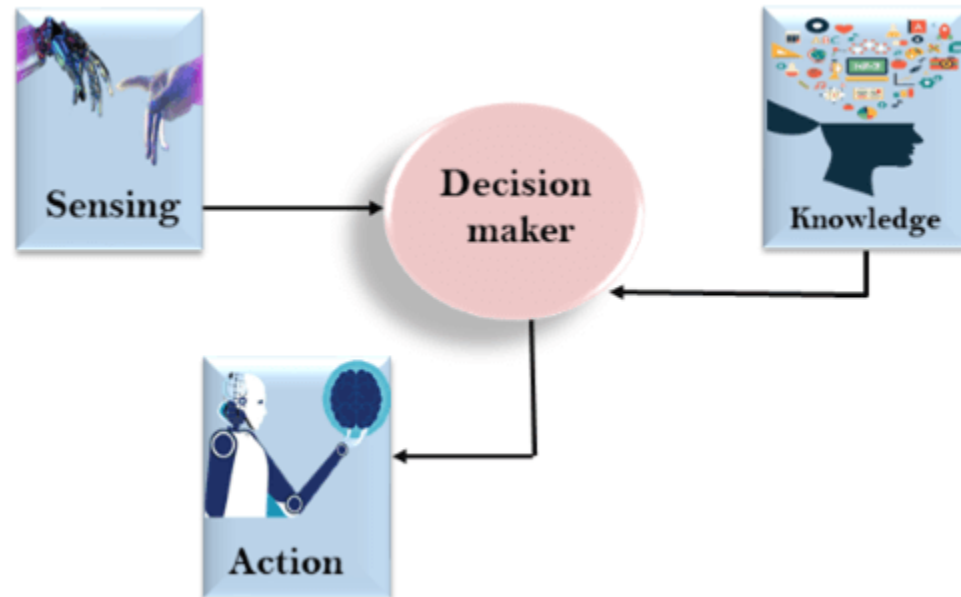
- Structural knowledge is basic knowledge to problem-solving.
- It describes relationships between various concepts such as kind of, part of, and grouping of something.
- It describes the relationship that exists between concepts or objects.

The relation between knowledge and intelligence:

Knowledge of real-worlds plays a vital role in intelligence and same for creating artificial intelligence. Knowledge plays an important role in demonstrating intelligent behavior in AI agents. An agent is only able to accurately act on some input when he has some knowledge or experience about that input.

Let's suppose if you met some person who is speaking in a language which you don't know, then how you will be able to act on that. The same thing applies to the intelligent behavior of the agents.

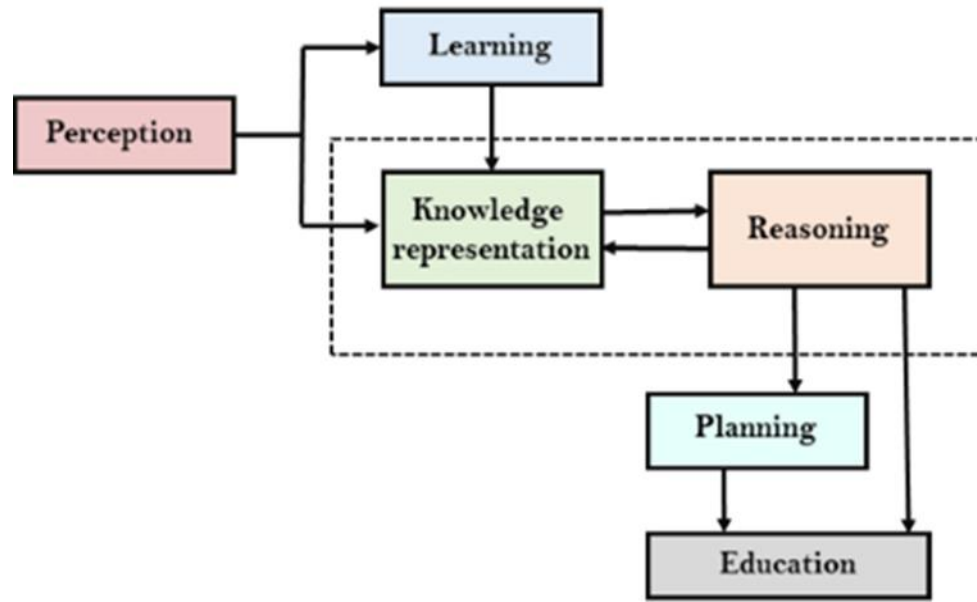
As we can see in below diagram, there is one decision maker which acts by sensing the environment and using knowledge. But if the knowledge part will not be present then, it cannot display intelligent behavior.



AI knowledge cycle:

An Artificial intelligence system has the following components for displaying intelligent behavior:

1. Perception
2. Learning
3. Knowledge Representation and Reasoning
4. Planning
5. Execution



The above diagram is showing how an AI system can interact with the real world and what components help it to show intelligence. AI system has Perception component by which it retrieves information from its environment. It can be visual, audio or another form of sensory input. The learning component is responsible for learning from data captured by Perception component. In the complete cycle, the main components are knowledge representation and Reasoning. These two components are involved in showing the intelligence in machine-like humans. These two components are independent with each other but also coupled together. The planning and execution depend on analysis of Knowledge representation and reasoning.

Approaches to knowledge representation:

There are mainly four approaches to knowledge representation, which are given below:

1. Simple relational knowledge:

- It is the simplest way of storing facts which uses the relational method, and each fact about a set of the object is set out systematically in columns.
- This approach of knowledge representation is famous in database systems where the relationship between different entities is represented.
- This approach has little opportunity for inference.

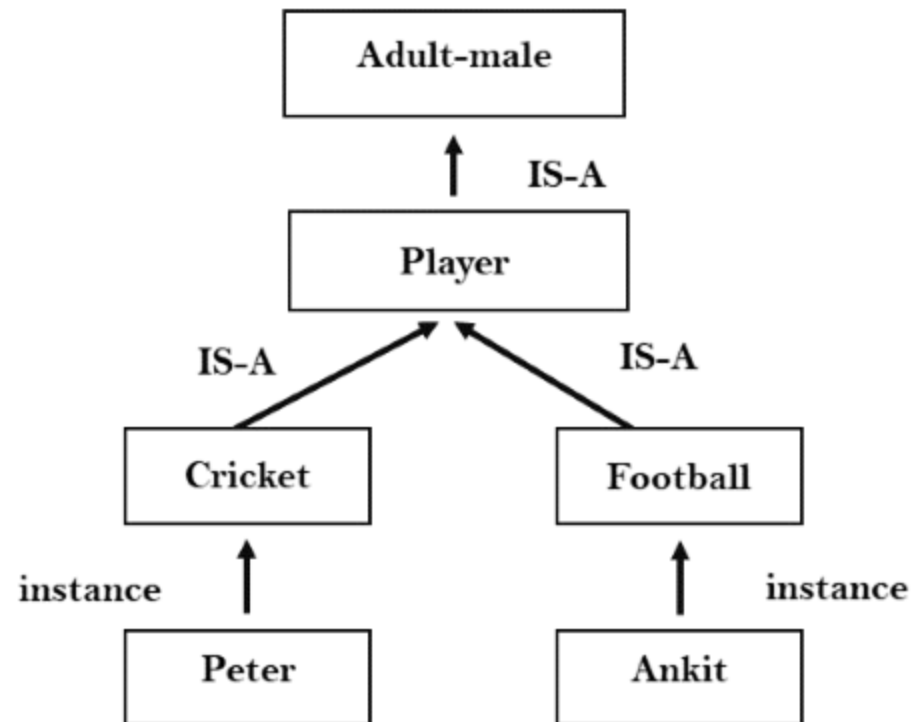
Example: The following is the simple relational knowledge representation.

Player	Weight	Age
Player1	65	23
Player2	58	18
Player3	75	24

2. Inheritable knowledge:

- In the inheritable knowledge approach, all data must be stored into a hierarchy of classes.
- All classes should be arranged in a generalized form or a hierarchal manner.
- In this approach, we apply inheritance property.
- Elements inherit values from other members of a class.
- This approach contains inheritable knowledge which shows a relation between instance and class, and it is called instance relation.
- Every individual frame can represent the collection of attributes and its value.
- In this approach, objects and values are represented in Boxed nodes.
- We use Arrows which point from objects to their values.

Example:



3. Inferential knowledge:

- Inferential knowledge approach represents knowledge in the form of formal logics.
- This approach can be used to derive more facts.
- It guaranteed correctness.
- **Example:** Let's suppose there are two statements:
 - Marcus is a man
 - All men are mortal

Then it can represent as;

man(Marcus)

$\forall x = \text{man}(x) \text{ -----} \rightarrow \text{mortal}(x)$

4. Procedural knowledge:

- Procedural knowledge approach uses small programs and codes which describes how to do specific things, and how to proceed.
- In this approach, one important rule is used which is **If-Then rule**.
- In this knowledge, we can use various coding languages such as **LISP language** and **Prolog language**.
- We can easily represent heuristic or domain-specific knowledge using this approach.
- But it is not necessary that we can represent all cases in this approach.

Requirements for knowledge Representation system:

A good knowledge representation system must possess the following properties.

1.Representational Accuracy:

KR system should have the ability to represent all kind of required knowledge.

2. Inferential Adequacy:

KR system should have ability to manipulate the representational structures to produce new knowledge corresponding to existing structure.

3. Inferential Efficiency:

The ability to direct the inferential knowledge mechanism into the most productive directions by storing appropriate guides.

4. Inquisitional efficiency- The ability to acquire the new knowledge easily using automatic methods.

Propositional logic: Propositional logic is a branch of mathematics that studies the logical relationships between propositions (or statements, sentences, assertions) taken as a whole, and connected via logical connectives.

The syntax of propositional logic defines the allowable sentences for the knowledge representation. There are two types of Propositions:

- **Atomic Propositions**
- **Compound propositions**

Atomic Proposition: Atomic propositions are the simple propositions. It consists of a single proposition symbol. These are the sentences which must be either true or false.

Example:

- a) $2+2$ is 4, it is an atomic proposition as it is a **true** fact.
- b) "The Sun is cold" is also a proposition as it is a **false** fact.

Compound proposition: Compound propositions are constructed by combining simpler or atomic propositions, using parenthesis and logical connectives.

Example:

- a) "It is raining today, and street is wet."
- b) "Ankit is a doctor, and his clinic is in Mumbai."

Logical Connectives:

Logical connectives are used to connect two simpler propositions or representing a sentence logically. We can create compound propositions with the help of logical connectives. There are mainly five connectives, which are given as follows:

1. **Negation:** A sentence such as $\neg P$ is called negation of P. A literal can be either Positive literal or negative literal.
2. **Conjunction:** A sentence which has $\wedge$ connective such as, $P \wedge Q$ is called a conjunction.

Example: Rohan is intelligent and hardworking. It can be written as,

P= Rohan is intelligent,

Q= Rohan is hardworking. $\rightarrow P \wedge Q$.

3. **Disjunction:** A sentence which has $\vee$ connective, such as $P \vee Q$. is called disjunction, where P and Q are the propositions.

Example: "Ritika is a doctor or Engineer",

Here P= Ritika is Doctor. Q= Ritika is Doctor, so we can write it as **$P \vee Q$.**

1. **Implication:** A sentence such as $P \rightarrow Q$, is called an implication. Implications are also known as if-then rules. It can be represented as

If it is raining, then the street is wet.

Let P = It is raining, and Q = Street is wet, so it is represented as $P \rightarrow Q$

2. **Biconditional:** A sentence such as $P \Leftrightarrow Q$ is a **Biconditional sentence**,
example If I am breathing, then I am alive

P = I am breathing, Q = I am alive, it can be represented as $P \Leftrightarrow Q$.

Following is the summarized table for Propositional Logic Connectives

Connective symbols	Word	Technical term	Example
$\wedge$	AND	Conjunction	$A \wedge B$
$\vee$	OR	Disjunction	$A \vee B$
$\rightarrow$	Implies	Implication	$A \rightarrow B$
$\Leftrightarrow$	If and only if	Biconditional	$A \Leftrightarrow B$
$\neg$ or $\sim$	Not	Negation	$\neg A$ or $\neg B$

Truth Table:

In propositional logic, we need to know the truth values of propositions in all possible scenarios. We can combine all the possible combination with logical connectives, and the representation of these combinations in a tabular format is called **Truth table**.

The truth table is a systematic representation of all truth values of a logical expression. It consists of all the inputs and gives the required output. It takes a combination of inputs to get the desired output. Truth tables are mainly used in Boolean algebra so, a variable can take two values 0 or 1.

The truth table is primarily used in digital circuits where it is used to validate the output generated from the various input combinations of the logical expressions. It is also used in mathematics and other fields which use Boolean logic and digital logic.

Following are the truth table for all logical connectives:

For Negation:

P	$\neg P$
True	False
False	True

For Conjunction:

P	Q	$P \wedge Q$
True	True	True
True	False	False
False	True	False
False	False	False

For disjunction:

P	Q	$P \vee Q$
True	True	True
False	True	True
True	False	True
False	False	False

For Implication:

P	Q	$P \rightarrow Q$
True	True	True
True	False	False
False	True	True
False	False	True

For Biconditional:

P	Q	$P \leftrightarrow Q$
True	True	True
True	False	False
False	True	False
False	False	True

Truth table with three propositions:

We can build a proposition composing three propositions P, Q, and R. This truth table is made-up of 8n Tuples as we have taken three proposition symbols.

P	Q	R	$\neg R$	$P \vee Q$	$P \vee Q \rightarrow \neg R$
True	True	True	False	True	False
True	True	False	True	True	True
True	False	True	False	True	False
True	False	False	True	True	True
False	True	True	False	True	False
False	True	False	True	True	True
False	False	True	False	False	True
False	False	False	True	False	True

Precedence of connectives:

Just like arithmetic operators, there is a precedence order for propositional connectors or logical operators. This order should be followed while evaluating a propositional problem. Following is the list of the precedence order for operators:

Precedence	Operators
First Precedence	Parenthesis
Second Precedence	Negation
Third Precedence	Conjunction(AND)
Fourth Precedence	Disjunction(OR)
Fifth Precedence	Implication
Six Precedence	Conditional

Logical equivalence:

- Logical equivalence is one of the features of propositional logic. Two propositions are said to be logically equivalent if and only if the columns in the truth table are identical to each other.
- Let's take two propositions A and B, so for logical equivalence, we can write it as $A \Leftrightarrow B$. In below truth table we can see that column for $\neg A \vee B$ and $A \rightarrow B$, are identical hence A is Equivalent to B

A	B	$\neg A$	$\neg A \vee B$	$A \rightarrow B$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Properties of Operators:

- **Commutatively:**
 - $P \wedge Q = Q \wedge P$, or
 - $P \vee Q = Q \vee P$.
- **Associativity:**
 - $(P \wedge Q) \wedge R = P \wedge (Q \wedge R)$,
 - $(P \vee Q) \vee R = P \vee (Q \vee R)$
- **Identity element:**
 - $P \wedge \text{True} = P$,
 - $P \vee \text{True} = \text{True}$.
- **Distributive:**
 - $P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R)$.
 - $P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$.
- **DE Morgan's Law:**
 - $\neg (P \wedge Q) = (\neg P) \vee (\neg Q)$
 - $\neg (P \vee Q) = (\neg P) \wedge (\neg Q)$.
- **Double-negation elimination:**
 - $\neg (\neg P) = P$.

Limitations of Propositional logic:

- We cannot represent relations like ALL, some, or none with propositional logic. Example:
 - **All the girls are intelligent.**
 - **Some apples are sweet.**
- Propositional logic has limited expressive power.
- In propositional logic, we cannot describe statements in terms of their properties or logical relationships.

Well-formed Formulas (WFFs) of Propositional Logic

Any expression that obeys the syntactic rules of propositional logic is called a well-formed formula, or WFF. Well-Formed Formula(WFF) is an expression consisting of variables(capital letters), parentheses, and connective symbols. An expression is basically a combination of operands & operators and here operands and operators are the connective symbols.

Propositional logic uses a symbolic “language” to represent the logical structure, or form, of a compound proposition. Like any language, this symbolic language has rules of syntax- grammatical rules for putting symbols together in the right way. Any expression that obeys the syntactic rules of propositional logic is called a well-formed formula, or WFF.

Fortunately, the syntax of propositional logic is easy to learn. It has only three rules:

Any capital letter by itself is a WFF.

Any WFF can be prefixed with “ $\sim$ ”. (The result will be a WFF too.)

Any two WFFs can be put together with “ $\cdot$ ”, “ $\vee$ ”, “ $\supset$ ”, or “ $\equiv$ ” between them, enclosing the result in parentheses. (This will be a WFF too.)

Tautology

A proposition P is a tautology if it is true under all circumstances. It means it contains the only T in the final column of its truth table.

Example: Prove that the statement $(p \rightarrow q) \leftrightarrow (\sim q \rightarrow \sim p)$ is a tautology.

Solution: Make the truth table of the above statement

p	q	$p \rightarrow q$	$\sim q$	$\sim p$	$\sim q \rightarrow \sim p$	$(p \rightarrow q) \leftrightarrow (\sim q \rightarrow \sim p)$
T	T	T	F	F	T	T
T	F	F	T	F	F	T
F	T	T	F	T	T	T
F	F	T	T	T	T	T

As the final column contains all T's, so it is a tautology.

Predicate Logic

Predicate Logic deals with predicates, which are propositions, consist of variables. A predicate is an expression of one or more variables determined on some specific domain. A predicate with variables can be made a proposition by either authorizing a value to the variable or by quantifying the variable.

The following are some examples of predicates.

Consider $E(x, y)$ denote " $x = y$ "

Consider $X(a, b, c)$ denote " $a + b + c = 0$ "

Consider $M(x, y)$ denote " x is married to y ."

Quantifier:

The variable of predicates is quantified by quantifiers. There are two types of quantifier in predicate logic - Existential Quantifier and Universal Quantifier.

Existential Quantifier:

If $p(x)$ is a proposition over the universe U . Then it is denoted as $\exists x p(x)$ and read as "There exists at least one value in the universe of variable x such that $p(x)$ is true.

The quantifier $\exists$ is called the existential quantifier.

There are several ways to write a proposition, with an existential quantifier, i.e.,

$(\exists x \in A)p(x)$ or $\exists x \in A$ such that $p(x)$ or $(\exists x)p(x)$ or $p(x)$ is true for some $x \in A$.

Universal Quantifier:

If $p(x)$ is a proposition over the universe U . Then it is denoted as $\forall x, p(x)$ and read as "For every $x \in U, p(x)$ is true." The quantifier $\forall$ is called the Universal Quantifier.

There are several ways to write a proposition, with a universal quantifier.

$\forall x \in A, p(x)$ or $p(x), \forall x \in A$ Or $\forall x, p(x)$ or $p(x)$ is true for all $x \in A$.

First-Order Logic in Artificial intelligence

In the topic of Propositional logic, we have seen that how to represent statements using propositional logic. But unfortunately, in propositional logic, we can only represent the facts, which are either true or false. PL is not sufficient to represent the complex sentences or natural language statements. The propositional logic has very limited expressive power. Consider the following sentence, which we cannot represent using PL logic.

"Some humans are intelligent", or

"Sachin likes cricket."

To represent the above statements, PL logic is not sufficient, so we required some more powerful logic, such as first-order logic.

- First-Order logic:
- First-order logic is another way of knowledge representation in artificial intelligence. It is an extension to propositional logic.
- FOL is sufficiently expressive to represent the natural language statements in a concise way.
- First-order logic is also known as **Predicate logic or First-order predicate logic**. First-order logic is a powerful language that develops information about the objects in a more easy way and can also express the relationship between those objects.
- First-order logic (like natural language) does not only assume that the world contains facts like propositional logic but also assumes the following things in the world:
 - **Objects:** A, B, people, numbers, colors, wars, theories, squares, pits, wumpus,
 - **Relations:** It can be unary relation such as: red, round, is adjacent, or n-any relation such as: the sister of, brother of, has color, comes between
 - **Function:** Father of, best friend, third inning of, end of,
- As a natural language, first-order logic also has two main parts:
 - **Syntax**
 - **Semantics**

Syntax of First-Order logic:

- The syntax of FOL determines which collection of symbols is a logical expression in first-order logic. The basic syntactic elements of first-order logic are symbols. We write statements in short-hand notation in FOL.
- Basic Elements of First-order logic:

Following are the basic elements of FOL syntax:

Constant	1, 2, A, John, Mumbai, cat,....
Variables	x, y, z, a, b,....
Predicates	Brother, Father, >,....
Function	sqrt, LeftLegOf,
Connectives	$\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\Leftrightarrow$
Equality	$=$
Quantifier	$\forall$, $\exists$

Atomic sentences:

Atomic sentences are the most basic sentences of first-order logic. These sentences are formed from a predicate symbol followed by a parenthesis with a sequence of terms.

We can represent atomic sentences as Predicate (term1, term2,, term n).

Example: Ravi and Ajay are brothers: $\Rightarrow$ Brothers(Ravi, Ajay).

Chinky is a cat: $\Rightarrow$ cat (Chinky).

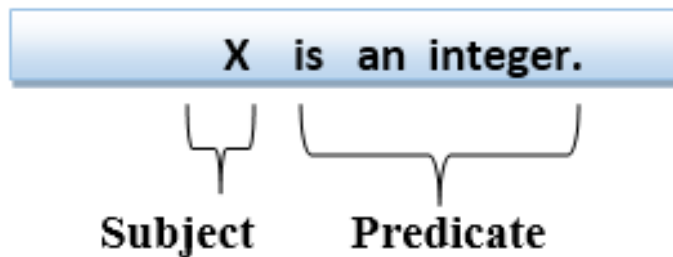
Complex Sentences:

Complex sentences are made by combining atomic sentences using connectives. First-order logic statements can be divided into two parts:

Subject: Subject is the main part of the statement.

Predicate: A predicate can be defined as a relation, which binds two atoms together in a statement.

Consider the statement: "x is an integer." it consists of two parts, the first part x is the subject of the statement and second part "is an integer," is known as a predicate.



Quantifiers in First-order logic:

A quantifier is a language element which generates quantification, and quantification specifies the quantity of specimen in the universe of discourse.

These are the symbols that permit to determine or identify the range and scope of the variable in the logical expression. There are two types of quantifier:

Universal Quantifier, (for all, everyone, everything)

Existential quantifier, (for some, at least one).

Universal Quantifier:

- Universal quantifier is a symbol of logical representation, which specifies that the statement within its range is true for everything or every instance of a particular thing. The Universal quantifier is represented by a symbol $\forall$, which resembles an inverted A.

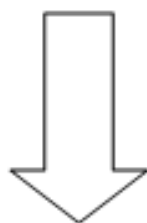
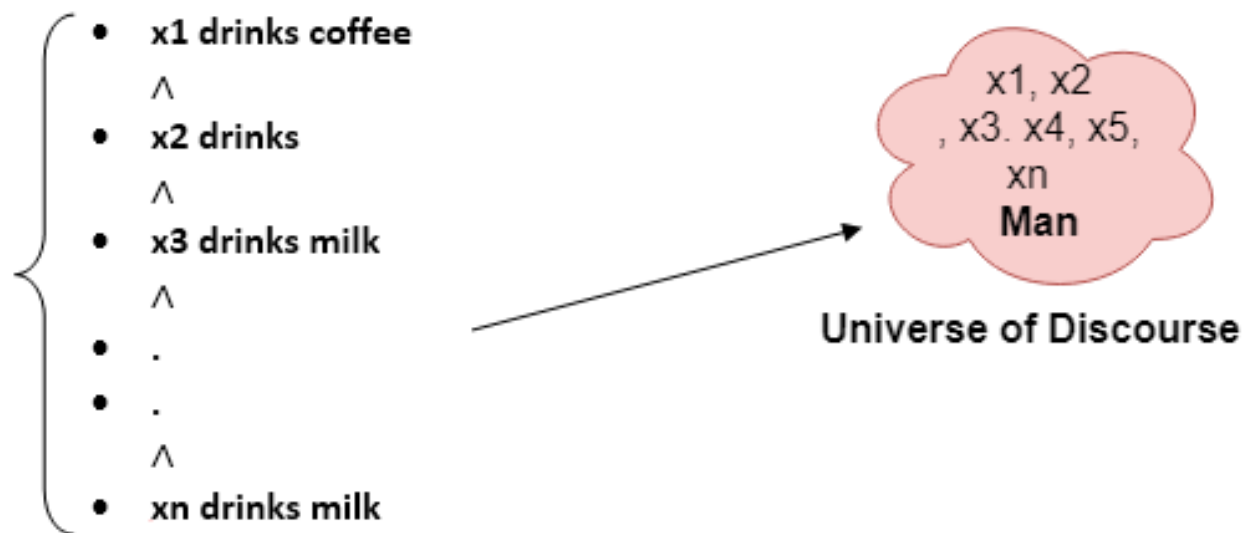
If x is a variable, then $\forall x$ is read as:

- **For all x**
- **For each x**
- **For every x.**

Example:

All man drink coffee.

Let a variable x which refers to a cat so all x can be represented in UOD as below:



So in shorthand notation, we can write it as :

- $\forall x \text{ man}(x) \rightarrow \text{drink}(x, \text{coffee}).$
- It will be read as: There are all x where x is a man who drink coffee.

Existential Quantifier:

- Existential quantifiers are the type of quantifiers, which express that the statement within its scope is true for at least one instance of something.
- It is denoted by the logical operator $\exists$, which resembles as inverted E. When it is used with a predicate variable then it is called as an existential quantifier.

If x is a variable, then existential quantifier will be $\exists x$ or $\exists(x)$. And it will be read as:

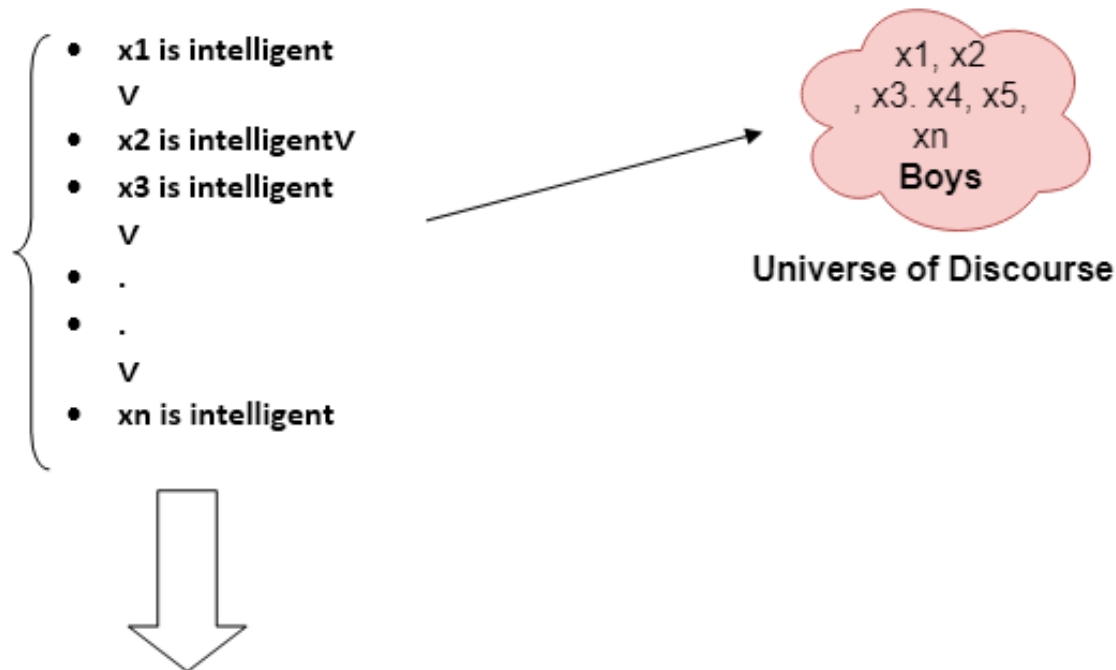
- **There exists a 'x.'**
- **For some 'x.'**
- **For at least one 'x.'**

Example:

Some boys are intelligent.

$\exists x: \text{boys}(x) \wedge \text{intelligent}(x)$

It will be read as: There are some x where x is a boy who is intelligent.



Points to remember:

- The main connective for universal quantifier $\forall$ is implication $\rightarrow$.
- The main connective for existential quantifier $\exists$ is and $\wedge$.

Properties of Quantifiers:

- In universal quantifier, $\forall x \forall y$ is similar to $\forall y \forall x$.
- In Existential quantifier, $\exists x \exists y$ is similar to $\exists y \exists x$.
- $\exists x \forall y$ is not similar to $\forall y \exists x$.

Some Examples of FOL using quantifier:

1. All birds fly.

In this question the predicate is "**fly(bird)**."

And since there are all birds who fly so it will be represented as follows.

$$\forall x \text{ bird}(x) \rightarrow \text{fly}(x).$$

2. Every man respects his parent.

In this question, the predicate is "**respect(x, y)**," where **x=man**, and **y= parent**.

Since there is every man so will use $\forall$, and it will be represented as follows:

$$\forall x \text{ man}(x) \rightarrow \text{respects}(x, \text{parent}).$$

Inference:

- In artificial intelligence, we need intelligent computers which can create new logic from old logic or by evidence, **so generating the conclusions from evidence and facts is termed as Inference.**

Inference rules:

- Inference rules are the templates for generating valid arguments. Inference rules are applied to derive proofs in artificial intelligence, and the proof is a sequence of conclusion that leads to the desired goal.
- In inference rules, the implication among all the connectives plays an important role.
- Inference rules are logical forms or guides that consist of premises and a conclusion. They are used to deduce valid arguments from given statements. There are different types of inference rules in AI, such as modus ponens, modus tollens, hypothetical syllogism, disjunctive syllogism, addition, simplification, and resolution. These rules have different forms and applications depending on the structure and content of the statements.
- Following are some terminologies related to inference rules:

Following are some terminologies related to inference rules:

Implication: It is one of the logical connectives which can be represented as $P \rightarrow Q$. It is a Boolean expression.

Converse: The converse of implication, which means the right-hand side proposition goes to the left-hand side and vice-versa. It can be written as $Q \rightarrow P$.

Contrapositive: The negation of converse is termed as contrapositive, and it can be represented as $\neg Q \rightarrow \neg P$.

Inverse: The negation of implication is called inverse. It can be represented as $\neg P \rightarrow \neg Q$.

From the above term some of the compound statements are equivalent to each other, which we can prove using truth table:

P	Q	$P \rightarrow Q$	$Q \rightarrow P$	$\neg Q \rightarrow \neg P$	$\neg P \rightarrow \neg Q$
T	T	T	T	T	T
T	F	F	T	F	T
F	T	T	F	T	F
F	F	T	T	T	T

Hence from the above truth table, we can prove that $P \rightarrow Q$ is equivalent to $\neg Q \rightarrow \neg P$, and $Q \rightarrow P$ is equivalent to $\neg P \rightarrow \neg Q$.

Types of Inference rules:

1. Modus Ponens:

The Modus Ponens rule is one of the most important rules of inference, and it states that if P and $P \rightarrow Q$ is true, then we can infer that Q will be true. It can be represented as:

$$\text{Notation for Modus ponens: } \frac{P \rightarrow Q, P}{\therefore Q}$$

Example:

Statement-1: "If I am sleepy then I go to bed" $\implies P \rightarrow Q$

Statement-2: "I am sleepy" $\implies P$

Conclusion: "I go to bed." $\implies Q$.

Hence, we can say that, if $P \rightarrow Q$ is true and P is true then Q will be true.

Proof by Truth table:

P	Q	$P \rightarrow Q$
0	0	0
0	1	1
1	0	0
1	1	1



Types of Inference rules:

2. Modus Tollens:

The Modus Tollens rule state that if $P \rightarrow Q$ is true and $\neg Q$ is true, then $\neg P$ will also true. It can be represented as:

$$\text{Notation for Modus Tollens: } \frac{P \rightarrow Q, \neg Q}{\neg P}$$

Statement-1: "If I am sleepy then I go to bed" $\implies P \rightarrow Q$

Statement-2: "I do not go to the bed." $\implies \neg Q$

Statement-3: Which infers that "**I am not sleepy**" $\implies \neg P$

Proof by Truth table:

P	Q	$\neg P$	$\neg Q$	$P \rightarrow Q$
0	0	1	1	1
0	1	1	0	1
1	0	0	1	0
1	1	0	0	1



Types of Inference rules:

3. Hypothetical Syllogism:

The Hypothetical Syllogism rule state that if $P \rightarrow R$ is true whenever $P \rightarrow Q$ is true, and $Q \rightarrow R$ is true. It can be represented as the following notation:

Example:

Statement-1: If you have my home key then you can unlock my home. $P \rightarrow Q$

Statement-2: If you can unlock my home then you can take my money. $Q \rightarrow R$

Conclusion: If you have my home key then you can take my money. $P \rightarrow R$

Proof by truth table:

P	Q	R	$P \rightarrow Q$	$Q \rightarrow R$	$P \rightarrow R$	
0	0	0	1	1	1	←
0	0	1	1	1	1	←
0	1	0	1	0	1	
0	1	1	1	1	1	←
1	0	0	0	1	1	
1	0	1	0	1	1	
1	1	0	1	0	0	
1	1	1	1	1	1	←

Types of Inference rules:

4. Disjunctive Syllogism:

The Disjunctive syllogism rule state that if $P \vee Q$ is true, and $\neg P$ is true, then Q will be true. It can be represented as:

$$\text{Notation of Disjunctive syllogism: } \frac{P \vee Q, \neg P}{Q}$$

Example:

Statement-1: Today is Sunday or Monday. $\implies P \vee Q$

Statement-2: Today is not Sunday. $\implies \neg P$

Conclusion: Today is Monday. $\implies Q$

Proof by truth-table:

P	Q	$\neg P$	$P \vee Q$
0	0	1	0
0	1	1	1
1	0	0	1
1	1	0	1



Types of Inference rules:

5. Addition:

The Addition rule is one the common inference rule, and it states that If P is true, then $P \vee Q$ will be true.

$$\text{Notation of Addition: } \frac{P}{P \vee Q}$$

Example

Statement: I have a vanilla ice-cream. $\implies P$

Statement-2: I have Chocolate ice-cream.

Conclusion: I have vanilla or chocolate ice-cream. $\implies (P \vee Q)$

Proof by Truth-Table:

P	Q	$P \vee Q$
0	0	0
1	0	1
0	1	1
1	1	1

Types of Inference rules:

6. Simplification:

The simplification rule state that if **$P \wedge Q$** is true, then **Q or P** will also be true. It can be represented as:

Notation of Simplification rule: $\frac{P \wedge Q}{Q}$ Or $\frac{P \wedge Q}{P}$

Proof by Truth-Table:

P	Q	$P \wedge Q$
0	0	0
1	0	0
0	1	0
1	1	1



Types of Inference rules:

7. Resolution:

The Resolution rule state that if $P \vee Q$ and $\neg P \wedge R$ is true, then $Q \vee R$ will also be true. **It can be represented as:**

Notation of Resolution

$$\frac{P \vee Q, \neg P \wedge R}{Q \vee R}$$

Proof by Truth-Table:

P	$\neg P$	Q	R	$P \vee Q$	$\neg P \wedge R$	$Q \vee R$
0	1	0	0	0	0	0
0	1	0	1	0	0	1
0	1	1	0	1	1	1
0	1	1	1	1	1	1
1	0	0	0	1	0	0
1	0	0	1	1	0	1
1	0	1	0	1	0	1
1	0	1	1	1	0	1

Resolution in FOPL

Resolution

- Resolution is a theorem proving technique that proceeds by building refutation proofs, i.e., proofs by contradictions. It was invented by a Mathematician John Alan Robinson in the year 1965.
- Resolution is used, if there are various statements are given, and we need to prove a conclusion of those statements. Unification is a key concept in proofs by resolutions. Resolution is a single inference rule which can efficiently operate on the **conjunctive normal form or clausal form**.
- **Clause:** Disjunction of literals (an atomic sentence) is called a **clause**. It is also known as a unit clause.
- **Conjunctive Normal Form:** A sentence represented as a conjunction of clauses is said to be **conjunctive normal form or CNF**.

Resolution in FOPL

The resolution inference rule:

The resolution rule for first-order logic is simply a lifted version of the propositional rule. Resolution can resolve two clauses if they contain complementary literals, which are assumed to be standardized apart so that they share no variables.

$$\frac{l_1 \vee \dots \vee l_k \quad m_1 \vee \dots \vee m_n}{\text{SUBST}(\theta, l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)}$$

Where l_i and m_j are complementary literals.

This rule is also called the **binary resolution rule** because it only resolves exactly two literals.

Example:

We can resolve two clauses which are given below:

[Animal (g(x) $\vee$ Loves (f(x), x)] and [$\neg$ Loves(a, b) $\vee$ $\neg$ Kills(a, b)]

Where two complimentary literals are: **Loves (f(x), x) and $\neg$ Loves (a, b)**

Resolution in FOPL

These literals can be unified with unifier $\theta = [a/f(x), \text{ and } b/x]$, and it will generate a resolvent clause:

$[\text{Animal}(g(x)) \vee \neg \text{Kills}(f(x), x)]$.

Steps for Resolution:

- Conversion of facts into first-order logic.
- Convert FOL statements into CNF
- Negate the statement which needs to prove (proof by contradiction)
- Draw resolution graph (unification).

Forward Chaining and Backward Chaining

Inference engine:

The inference engine is the component of the intelligent system in artificial intelligence, which applies logical rules to the knowledge base to infer new information from known facts. The first inference engine was part of the expert system. Inference engine commonly proceeds in two modes, which are:

i) **Forward chaining**

ii) **Backward chaining**

Horn Clause and Definite clause:

- Horn clause and definite clause are the forms of sentences, which enables knowledge base to use a more restricted and efficient inference algorithm. Logical inference algorithms use forward and backward chaining approaches, which require KB in the form of the **first-order definite clause**.

Definite clause: A clause which is a disjunction of literals with **exactly one positive literal** is known as a definite clause or strict horn clause.

Horn clause: A clause which is a disjunction of literals with **at most one positive literal** is known as horn clause. Hence all the definite clauses are horn clauses.

Example: $(\neg p \vee \neg q \vee k)$. It has only one positive literal k.

A. Forward Chaining: Forward chaining is also known as a forward deduction or forward reasoning method when using an inference engine. Forward chaining is a form of reasoning which start with atomic sentences in the knowledge base and applies inference rules (Modus Ponens) in the forward direction to extract more data until a goal is reached.

- The Forward-chaining algorithm starts from known facts, triggers all rules whose premises are satisfied, and add their conclusion to the known facts. This process repeats until the problem is solved.

Properties of Forward-Chaining:

- It is a down-up approach, as it moves from bottom to top.
- It is a process of making a conclusion based on known facts or data, by starting from the initial state and reaches the goal state.
- Forward-chaining approach is also called as data-driven as we reach to the goal using available data.
- Forward -chaining approach is commonly used in the expert system, such as CLIPS, business, and production rule systems.

B. Backward Chaining:

- Backward-chaining is also known as a backward deduction or backward reasoning method when using an inference engine. A backward chaining algorithm is a form of reasoning, which starts with the goal and works backward, chaining through rules to find known facts that support the goal.

Properties of backward chaining:

- It is known as a top-down approach.
- Backward-chaining is based on modus ponens inference rule.
- In backward chaining, the goal is broken into sub-goal or sub-goals to prove the facts true.
- It is called a goal-driven approach, as a list of goals decides which rules are selected and used.
- Backward -chaining algorithm is used in game theory, automated theorem proving tools, inference engines, proof assistants, and various AI applications.
- The backward-chaining method mostly used a **depth-first search** strategy for proof.

S. No.	Forward Chaining	Backward Chaining
1.	Forward chaining starts from known facts and applies inference rule to extract more data unit it reaches to the goal.	Backward chaining starts from the goal and works backward through inference rules to find the required facts that support the goal.
2.	It is a bottom-up approach	It is a top-down approach
3.	Forward chaining is known as data-driven inference technique as we reach to the goal using the available data.	Backward chaining is known as goal-driven technique as we start from the goal and divide into sub-goal to extract the facts.
4.	Forward chaining reasoning applies a breadth-first search strategy.	Backward chaining reasoning applies a depth-first search strategy.
5.	Forward chaining tests for all the available rules	Backward chaining only tests for few required rules.
6.	Forward chaining is suitable for the planning, monitoring, control, and interpretation application.	Backward chaining is suitable for diagnostic, prescription, and debugging application.
7.	Forward chaining can generate an infinite number of possible conclusions.	Backward chaining generates a finite number of possible conclusions.
8.	It operates in the forward direction.	It operates in the backward direction.
9.	Forward chaining is aimed for any conclusion.	Backward chaining is only aimed for the required data.

Semantic Network Representation

Semantic networks are alternative of predicate logic for knowledge representation. In Semantic networks, we can represent our knowledge in the form of graphical networks. This network consists of nodes representing objects and arcs which describe the relationship between those objects. Semantic networks can categorize the object in different forms and can also link those objects. Semantic networks are easy to understand and can be easily extended.

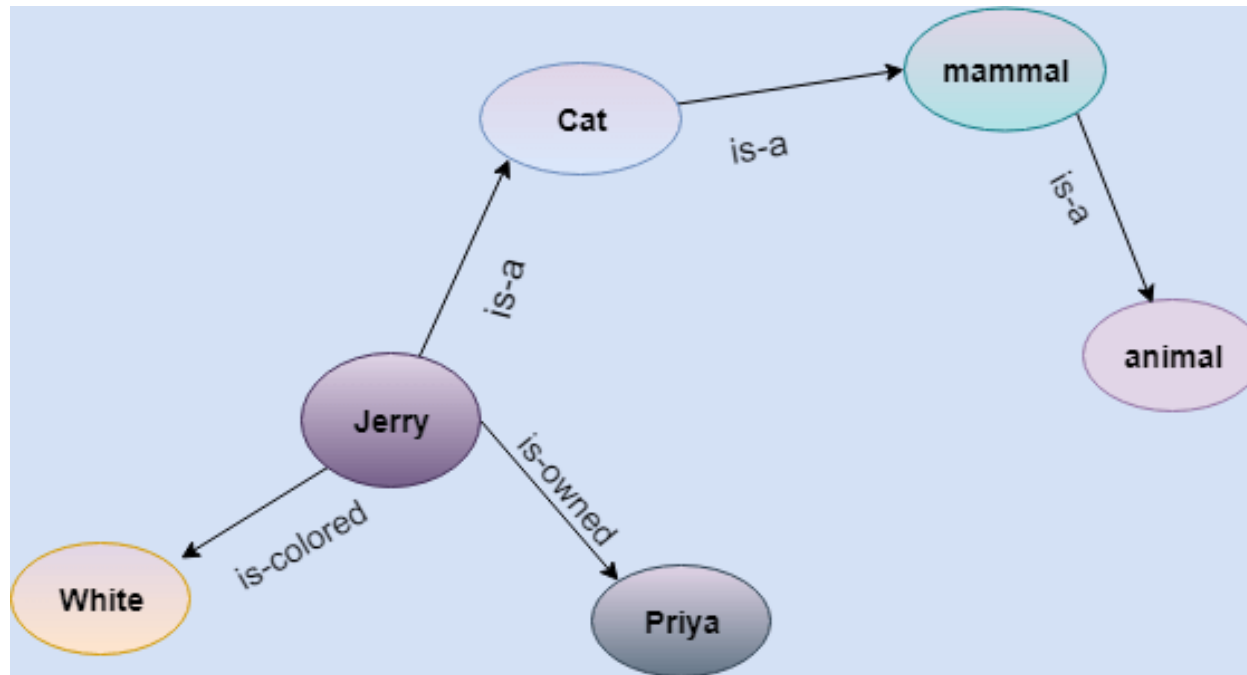
This representation consist of mainly two types of relations:

- a. IS-A relation (Inheritance)
- b. Kind-of-relation

Example: Following are some statements which we need to represent in the form of nodes and arcs. Statements:

- a. Jerry is a cat.
- b. Jerry is a mammal
- c. Jerry is owned by Priya.
- d. Jerry is brown colored.
- e. All Mammals are animal.

In the below diagram, we have represented the different type of knowledge in the form of nodes and arcs. Each object is connected with another object by some relation.



Advantages of Semantic network:

- 1.Semantic networks are a natural representation of knowledge.
- 2.Semantic networks convey meaning in a transparent manner.
- 3.These networks are simple and easily understandable.

Frame Representation

- A frame is a record like structure which consists of a collection of attributes and its values to describe an entity in the world. Frames are the AI data structure which divides knowledge into substructures by representing stereotypes situations. It consists of a collection of slots and slot values. These slots may be of any type and sizes. Slots have names and values which are called facets.
- **Facets:** The various aspects of a slot is known as **Facets**. Facets are features of frames which enable us to put constraints on the frames. Example: IF-NEEDED facts are called when data of any particular slot is needed. A frame may consist of any number of slots, and a slot may include any number of facets and facets may have any number of values. A frame is also known as **slot-filter knowledge representation** in artificial intelligence.
- Frames are derived from semantic networks and later evolved into our modern-day classes and objects. A single frame is not much useful. Frames system consist of a collection of frames which are connected. In the frame, knowledge about an object or event can be stored together in the knowledge base. The frame is a type of technology which is widely used in various applications including Natural language processing and machine visions.

Example: Let's suppose we are taking an entity, Peter. Peter is an engineer as a profession, and his age is 25, he lives in city London, and the country is England. So following is the frame representation for this:

Slots	Filter
Name	Peter
Profession	Doctor
Age	25
Marital status	Single
Weight	78

Advantages of frame representation:

- 1.The frame knowledge representation makes the programming easier by grouping the related data.
- 2.The frame representation is comparably flexible and used by many applications in AI.
- 3.It is very easy to add slots for new attribute and relations.
- 4.It is easy to include default data and to search for missing values.
- 5.Frame representation is easy to understand and visualize.

Uncertainty: Reasoning under uncertainty involves making rational decisions even when there is not enough information to prove that an action will work. Some reasons for reasoning under uncertainty include:

- True uncertainty (e.g., flipping a coin)
- Theoretical ignorance (no complete theory known about the problem domain, e.g., medical diagnosis)
- Laziness

Monotonic Reasoning

Monotonic Reasoning is the process that does not change its direction or can say that it moves in the one direction.

Monotonic Reasoning will move in the same direction continuously means it will either move in increasing order or decrease.

But since Monotonic Reasoning depends on knowledge and facts, It will only increase and will never decrease in this reasoning.

Example:

Sun rises in the East and sets in the West.

Statistical Reasoning: Statistical reasoning is a fundamental approach that harnesses the power of data to make informed decisions, predictions, and inferences. It is a first course in statistics for students whose college and career paths require knowledge of the fundamentals of the collection, analysis, and interpretation of data. Emphasis is placed on the development of statistical thinking, simulation, and the use of statistical software.

Statistical reasoning involves using statistical principles and methods to draw conclusions from data. It encompasses the following key components:

Data Collection: Gathering relevant data through surveys, experiments, or observational studies. The quality and method of data collection significantly impact the reliability of the conclusions.

Descriptive Statistics: Summarizing and describing the main features of a data set. This includes measures of central tendency (mean, median, mode) and measures of variability (range, variance, standard deviation).

Probability: Assessing the likelihood of different outcomes. Understanding probability helps in making predictions and inferences about a larger population based on sample data.

Inferential Statistics: Making predictions or inferences about a population based on a sample. This includes hypothesis testing, confidence intervals, and regression analysis.

Hypothesis Testing: Evaluating assumptions or claims about a population parameter. This involves setting up null and alternative hypotheses, selecting an appropriate test, and using a significance level to make a decision.

Confidence Intervals: Estimating a range within which a population parameter is likely to lie. Confidence intervals provide an indication of the precision of the estimate.

Regression Analysis: Exploring relationships between variables. Linear regression, for example, helps in understanding how the dependent variable changes with respect to one or more independent variables.

Correlation: Measuring the strength and direction of the relationship between two variables. Correlation coefficients range from -1 to 1, where values close to -1 or 1 indicate a strong relationship, and values close to 0 indicate a weak relationship.

Sampling: Selecting a subset of individuals from a population to estimate characteristics of the whole population. Good sampling methods ensure that the sample is representative and that the results are generalizable.

Bias and Variability: Recognizing and minimizing bias (systematic errors) and understanding variability (random errors) in data collection and analysis to improve the accuracy and reliability of conclusions.

Case-based reasoning (CBR): Case-based reasoning (CBR) is a problem-solving approach that leverages past experiences (cases) to solve new problems. Here's how it works:

Retrieve: The system retrieves relevant cases from its memory based on the current problem.

Reuse: It adapts and reuses solutions from similar cases.

Revise: If needed, the solution is revised to fit the current problem.

Retain: The updated solution is stored for future use.

Basis of CBR :

Here, the basis key parameters of CBR.

Regularity-

The identical steps executed under the same circumstances will tend to have the same or alike outcomes.

Typicality-

Experiences tend to repeat themselves.

Consistency-

Minor switch in the circumstances require merely small changes in the explanation and in the effect.

Adaptability-

When things replicate, the dissimilarities tend to be minute, and the small differences are uncomplicated to repay for.